

Complete Roadmap: Parliamentary AI Analyst

Project Overview

Aspect	Details
Project Name	Parliamentary AI Analyst
Problem Statement	Parliamentary information is vast, complex, and difficult for citizens, journalists, and MPs to navigate efficiently
Solution	AI-powered question-answering system that makes parliamentary data accessible through natural language
Goal	Win the UK Parliament Hackathon, create real-world impact, and build a stand-out project

Problem Statement (For Your Pitch)

"The UK Parliament produces thousands of pages of debates, written answers, and legislation every year. Citizens, journalists, and even MPs struggle to find specific information quickly. Our solution is an AI-powered question-answering system that makes parliamentary data accessible through simple natural language questions, improving transparency, accountability, and civic engagement."

Complete Roadmap (Day by Day)

17 Phase 1: Planning & Setup (Day 1 – 26 August)

Step	Task	Tech	Time	Where
1.1	Define project scope & features	—	1 hr	Planning
1.2	Set up GitHub repository	Git	30 min	GitHub.com
1.3	Create project structure	—	1 hr	Local
1.4	Set up Python virtual environment	Python	30 min	Local
1.5	Install dependencies	pip	30 min	Local
1.6	Set up React app	React, Vite	1 hr	Local
1.7	Install Tailwind CSS	Tailwind	30 min	Local
1.8	Create Figma/design mockup	Figma	2 hrs	Figma.com

End of Day 1: Project structure ready, GitHub repo set up, design mockup created

Phase 2: Data Collection & Processing (Day 2 – 27 August)

Step	Task	Tech	Time	Where
2.1	Research UK Parliament data sources	—	2 hrs	Online
2.2	Download Hansard data (debates)	Python, Requests	2 hrs	Local
2.3	Download Parliamentary Questions data	Python, Requests	2 hrs	Local
2.4	Download legislation data	Python, Requests	2 hrs	Local

Step	Task	Tech	Time	Where
2.5	Clean and preprocess data	Python, Pandas	3 hrs	Local
2.6	Create data loading script	Python	2 hrs	Local

End of Day 2: Data collected, cleaned, and ready for processing

Data Sources:

Source	URL	Data Type
Hansard	https://hansard.parliament.uk/	Parliamentary debates
Parliamentary Questions	https://www.parliament.uk/business/publications/written-questions-answers-statements/	Written questions
Legislation	https://www.legislation.gov.uk/	UK laws
Government Spending	https://www.gov.uk/government/statistics	Public spending

Phase 3: AI & Backend Development (Day 3-4 – 28-29 August)

Step	Task	Tech	Time	Where
3.1	Set up FastAPI backend	FastAPI, Python	2 hrs	Local
3.2	Create data processing pipeline	Python, Pandas	3 hrs	Local

Step	Task	Tech	Time	Where
3.3	Set up vector database (Pinecone/ChromaDB)	Pinecone/ChromaDB	2 hrs	Cloud/Local
3.4	Create embeddings	OpenAI/Cohere, LangChain	3 hrs	Cloud API
3.5	Build RAG pipeline	LangChain, RAG	4 hrs	Local
3.6	Create query engine	LangChain	3 hrs	Local
3.7	Build API endpoints	FastAPI	3 hrs	Local

End of Day 3-4: Backend API working with RAG pipeline

Code Structure:

```
python
# backend/app/main.py
from fastapi import FastAPI
from pydantic import BaseModel
from langchain.chains import RetrievalQA
from langchain.vectorstores import Pinecone
from langchain.embeddings import OpenAIEmbeddings

app = FastAPI(title="Parliamentary AI Analyst API")

class Question(BaseModel):
    query: str

@app.post("/ask")
def ask(question: Question):
    # RAG pipeline processes question
    answer = rag_engine.query(question.query)
    return {"answer": answer["result"], "sources": answer["source_documents"]}
```



Phase 4: Frontend Development (Day 5-6 – 30-31 August)

Step	Task	Tech	Time	Where
4.1	Create React components	React, JSX	4 hrs	Local
4.2	Build home page	React, Tailwind	3 hrs	Local
4.3	Build search bar	React	2 hrs	Local
4.4	Build results page	React, Tailwind	3 hrs	Local
4.5	Add API integration	React, Fetch/Axios	3 hrs	Local
4.6	Add loading states	React	2 hrs	Local
4.7	Make responsive	Tailwind	2 hrs	Local

End of Day 5-6: Fully functional React frontend connected to API

React Components:

```
jsx
// frontend/src/components/SearchBar.jsx
const SearchBar = ({ onSearch }) => {
  const [query, setQuery] = useState('');

  const handleSubmit = (e) => {
    e.preventDefault();
    onSearch(query);
  };

  return (
    <form onSubmit={handleSubmit} className="w-full max-w-3xl">
      <div className="flex gap-4">
        <input
          type="text"
          placeholder="What would you like to know?"
```

```

        className="flex-1 p-4 border-2 border-gray-200 rounded-xl focus:
border-primary focus:outline-none"
        value={query}
        onChange={(e) => setQuery(e.target.value)}
      />
      <button
        type="submit"
        className="px-8 py-4 bg-primary text-white rounded-xl font-semib
old hover:bg-primary/90"
      >
        Ask
      </button>
    </div>
  </form>
);
};

```

Phase 5: Testing & Polish (Day 7 – 1 September)

Step	Task	Tech	Time	Where
5.1	Test all functionality	Manual testing	2 hrs	Local
5.2	Fix bugs	—	3 hrs	Local
5.3	Improve UI/UX	Tailwind	3 hrs	Local
5.4	Add error handling	React, FastAPI	2 hrs	Local
5.5	Performance optimisation	—	2 hrs	Local

End of Day 7: Fully tested, bug-free application

Phase 6: Deployment (Day 8 – 2 September)

Step	Task	Tech	Time	Where
6.1	Deploy backend to Render	Render	2 hrs	Render.com
6.2	Deploy frontend to Vercel	Vercel	2 hrs	Vercel.com
6.3	Configure environment variables	—	1 hr	Render/Vercel
6.4	Test deployed application	—	2 hrs	Online
6.5	Create demo video	Screen recording	2 hrs	Local

End of Day 8: Live application accessible to everyone



Phase 7: Pitch Preparation (Day 9 – 3 September)

Step	Task	Tech	Time	Where
7.1	Create presentation slides	PowerPoint/Canva	3 hrs	Online
7.2	Write pitch script	—	2 hrs	Local
7.3	Practice pitch	—	2 hrs	Local
7.4	Prepare demo flow	—	2 hrs	Local
7.5	Prepare Q&A answers	—	2 hrs	Local

End of Day 9: Ready for the hackathon



Day 10 – Hackathon Day (4 September)

Step	Task	Time
8.1	Final testing	1 hr
8.2	Setup at venue	1 hr
8.3	Networking	1 hr
8.4	Pitch presentation	10 min
8.5	Demo	10 min
8.6	Q&A	10 min
8.7	Networking	2 hrs

Tech Stack Summary

Layer	Technology	Where
Frontend	React, Tailwind CSS, Vite	Local → Vercel
Backend	FastAPI, Python	Local → Render
AI/LLM	LangChain, OpenAI/Claude API, RAG	Cloud API
Vector DB	Pinecone or ChromaDB	Cloud/Local
Data	Pandas, NumPy, Requests	Local
Version Control	Git, GitHub	GitHub.com
Hosting	Vercel (frontend), Render (backend)	Cloud

Total Time Breakdown

Phase	Days	Hours	Focus
Planning & Setup	1	7 hrs	Project structure
Data Collection	1	14 hrs	Parliament data
Backend Development	2	18 hrs	API, RAG, AI
Frontend Development	2	18 hrs	React, UI
Testing & Polish	1	12 hrs	Bug fixes, UX
Deployment	1	8 hrs	Live hosting
Pitch Preparation	1	10 hrs	Presentation, practice
Total	10 days	~85 hrs	Full project

Project Structure

```
text
parliamentary-ai-analyst/
├── backend/
│   ├── app/
│   │   ├── main.py           # FastAPI server
│   │   ├── rag_engine.py     # LangChain RAG pipeline
│   │   ├── data_loader.py    # Load Parliament data
│   │   ├── embeddings.py     # Vector embeddings
│   │   └── config.py         # API keys & config
│   ├── data/
│   │   ├── raw/              # Raw data files
│   │   └── processed/        # Processed data
│   └── requirements.txt
└── frontend/
```

```
|   |— src/
|   |   |— components/
|   |   |   |— Header.jsx
|   |   |   |— SearchBar.jsx
|   |   |   |— ResultCard.jsx
|   |   |   |— StatsBanner.jsx
|   |   |   └─ Footer.jsx
|   |   |— pages/
|   |   |   |— Home.jsx
|   |   |   └─ Results.jsx
|   |   |— hooks/
|   |   |   └─ useParliamentAPI.js
|   |   |— styles/
|   |   |   └─ index.css
|   |   |— App.jsx
|   |   └─ main.jsx
|   |— package.json
|   └─ tailwind.config.js
|
|— pitch/
|   └─ presentation.pptx
|
|— README.md
└─ docker-compose.yml
```

Ready to Start? Let's Build!

Step	Action	Status
1	Create GitHub repository	☐ Not started
2	Set up project structure	☐ Not started
3	Build backend	☐ Not started
4	Build frontend	☐ Not started
5	Deploy	☐ Not started

Let me know which step you want to start with, and I'll give you the detailed code! 🚀